



Universidade de Brasília

Departamento de Ciência da Computação

Hierarquia de Memória



Memória Ideal

- Tamanho: Infinito
- Tempo de Acesso: Zero
- Custo: Zero

Impossível de ser realizada!

Logo, necessitamos de técnicas que criem a ilusão de termos esses requisitos.



Analogia do Patterson: (*mais uma! aff...*)

- Biblioteca
- Mesa
- Livro
- Seu foco de leitura

O tempo de acesso à informação está em ordem decrescente.

A quantidade de informação disponível está em ordem decrescente.

Se vc leu algo em um livro, muito provavelmente vai usá-lo novamente em pouco tempo.

Na biblioteca os livros com assuntos semelhantes estão localizados próximos.



■ **Princípio da Localidade Temporal:**

Se um item é referenciado, ele tenderá a ser referenciado novamente em breve.

- um livro lido, provavelmente será em breve reconsultado
- se uma instrução foi executada, provavelmente ela será reexecutada (loops).
- se um dado foi acessado, provavelmente o será novamente (leitura ou escrita)

■ **Princípio da Localidade Espacial:**

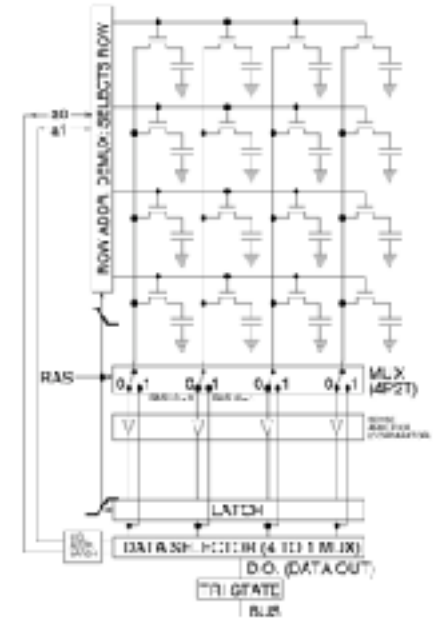
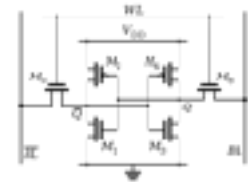
Se um item é referenciado, os itens próximos tenderão a ser referenciados em breve.

- os livros estão geralmente organizados por assunto nas prateleiras.
- os programas são geralmente sequenciais
- os dados são geralmente armazenados em estruturas (vetores, matrizes)

Hierarquia de Memória

- Considerando que:
 - Memórias mais rápidas são mais caras (\$\$\$) por bit
 - Memórias mais lentas são mais baratas(\$/2) por bit

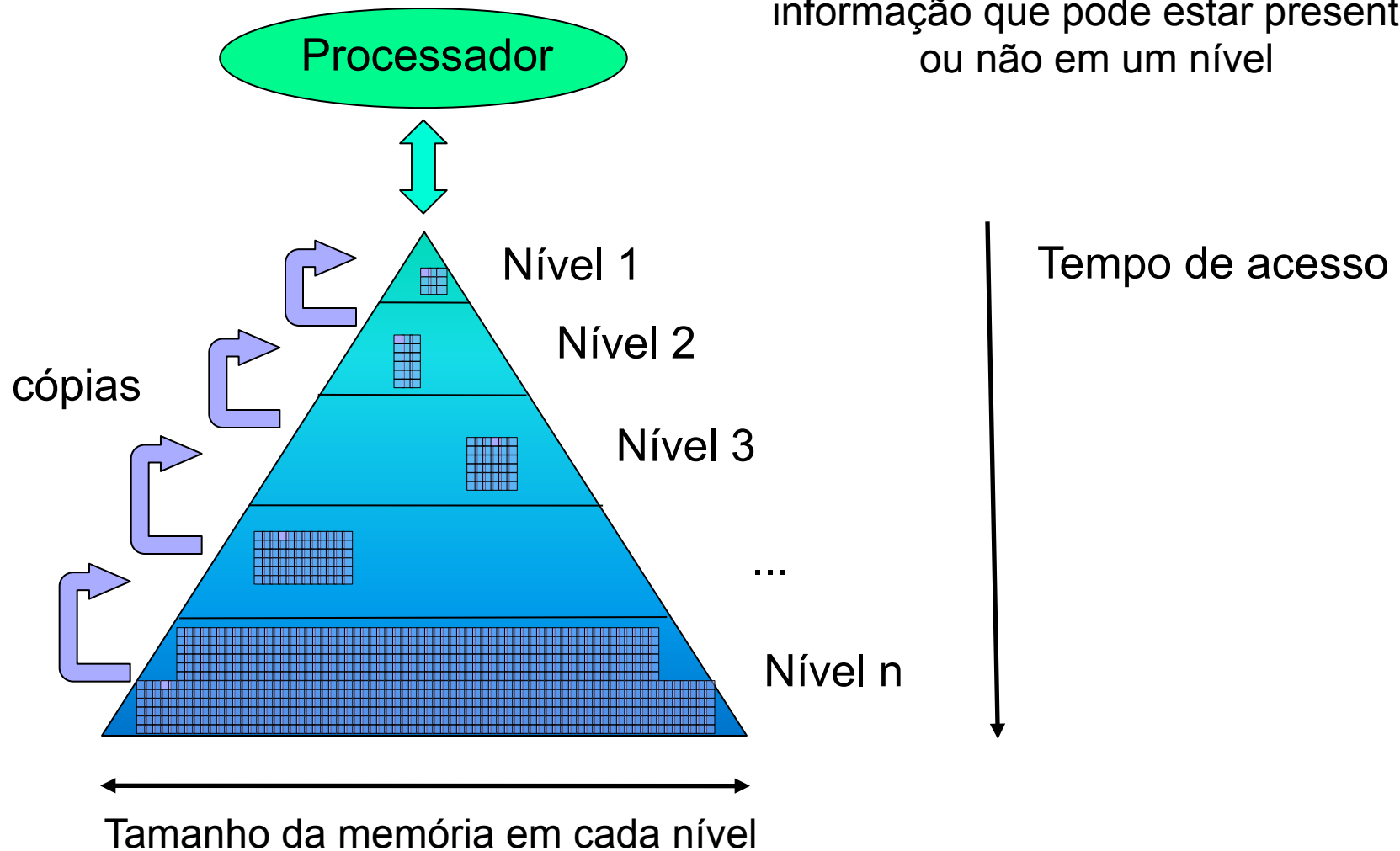
- Tecnologias mais populares atualmente
 - SRAM : Memória RAM estática
 - DRAM : Memória RAM dinâmica
 - Disco Magnético
 - Outras: Memórias Flash



Tecnologia	Tempo de acesso típico	\$ por GB
Static RAM (SRAM)	0,5 ~ 2,5 ns	\$2.000 ~ \$5.000
DRAM (DDR3 4GB)	10 ~20 ns	\$20 ~ \$75
FLASH (SSD 500GB)	20.000 ~ 50.000 ns	\$3 ~ \$10
HD (3TB)	5 ms ~ 20 ms	\$0,2 ~ \$2

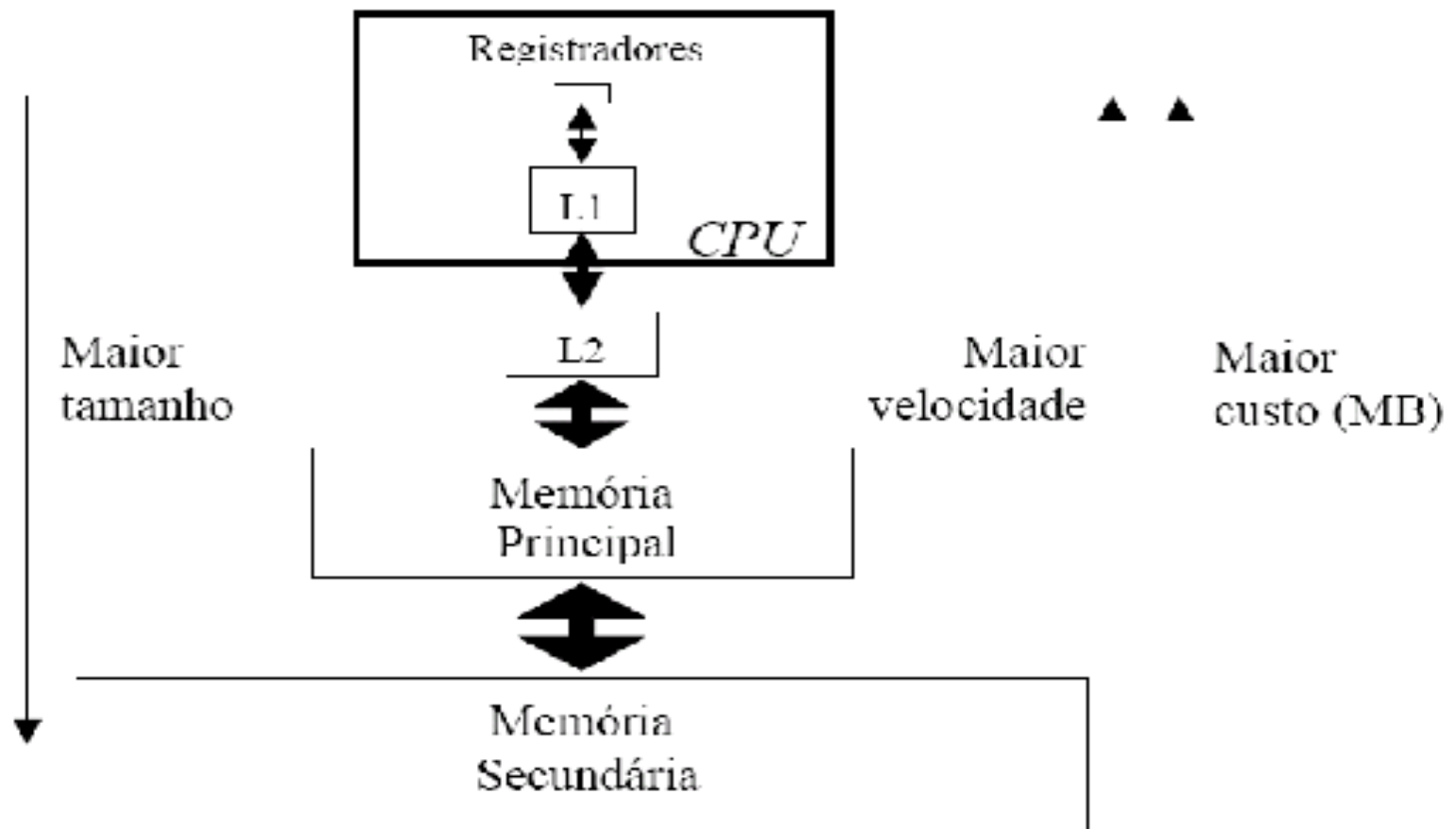
Hierarquia de Memória

Bloco: unidade mínima de informação que pode estar presente ou não em um nível

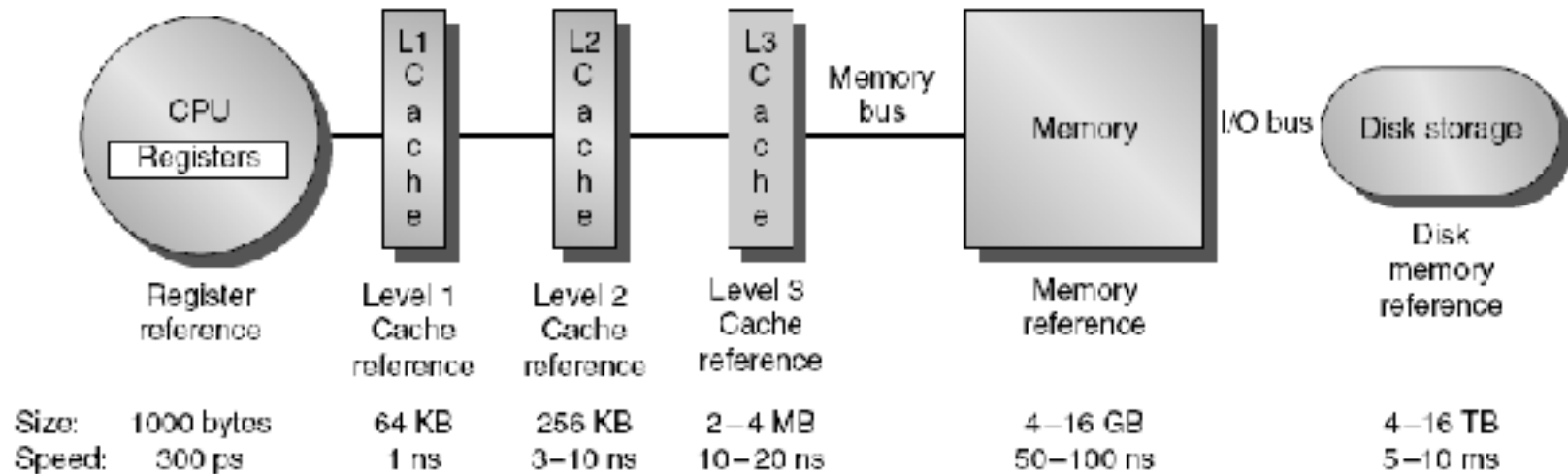


Níveis da Hierarquia de Memória

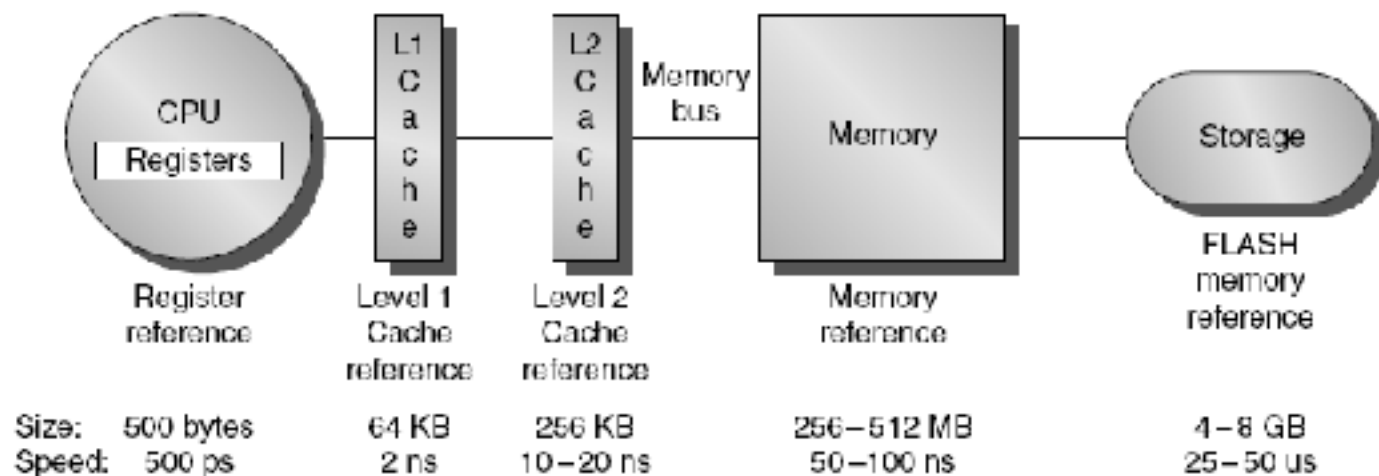
- Níveis intermediários usados para amortizar a diferença de velocidade entre processador e memória



Comparação de Características da Hierarquia de Memória

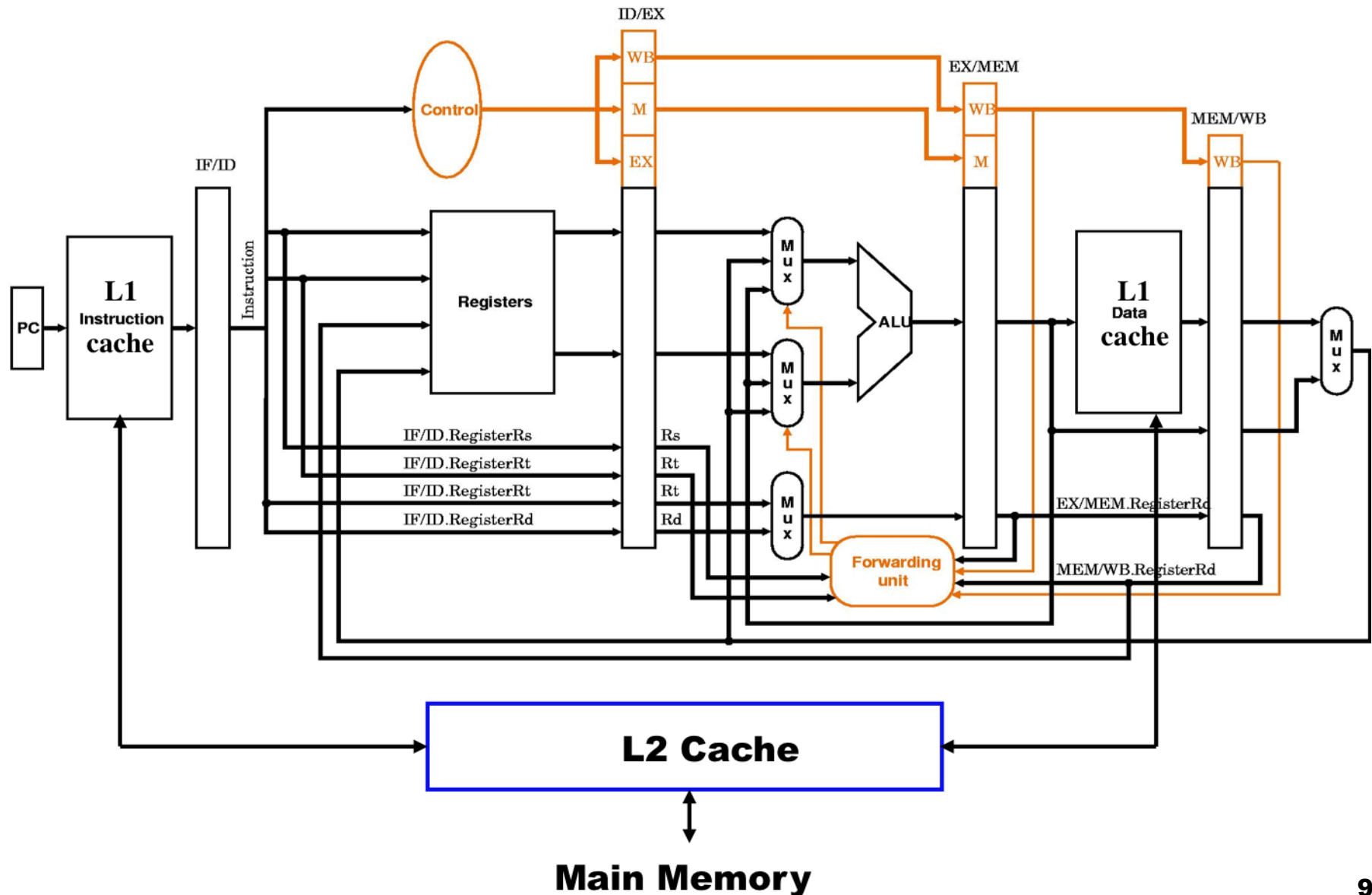


(a) Memory hierarchy for server



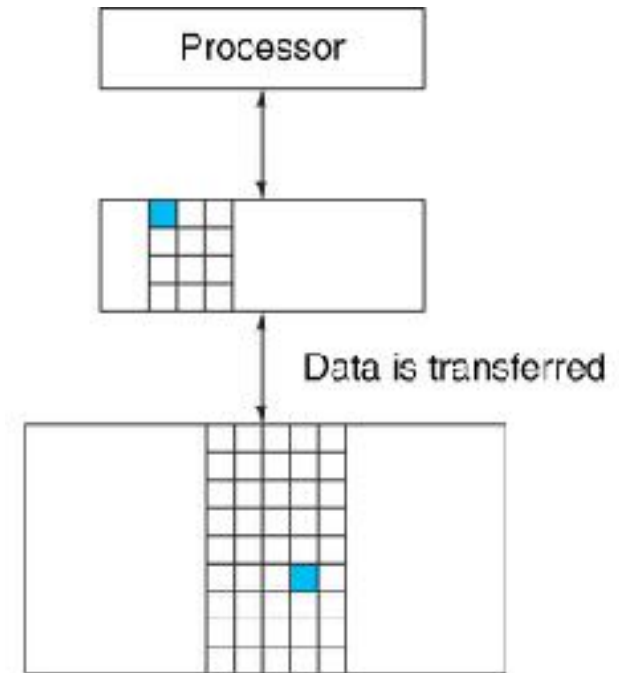
(b) Memory hierarchy for a personal mobile device

Cache no Pipeline RISC-V



Princípios Básicos de Cache

- Considerando apenas a relação entre 2 níveis da hierarquia.
 - **Bloco**: unidade mínima de dados
 - **Acerto (*hit*)** : dado encontra-se no nível superior
 - **Taxa acerto**: é a fração dos acessos à memória encontrados no nível superior.
 - **Taxa de Erro = (1-Taxa de acerto)**
 - **Tempo de acerto**: é o tempo necessário para acessar o nível superior, incluindo o tempo para decidir se é falha ou acerto.
 - **Penalidade de falha**: é o tempo para substituir um bloco no nível superior pelo bloco correspondente do nível inferior, mais o tempo para acessar este bloco.



Princípios Básicos de Cache

$$\text{Miss Rate} = \frac{\text{número de misses}}{\text{Número total de acessos de memória}} = 1 - \text{Hit Rate}$$

$$\text{Hit Rate} = \frac{\text{Número de hits}}{\text{Número total de acessos de memória}} = 1 - \text{Miss Rate}$$

Exemplo 8.1 CALCULANDO A PERFORMANCE DA CACHE

Suponha que um programa tem 2000 instruções de acesso de dados (carregamentos ou armazenamentos), e 1250 desses valores de dados solicitados são encontrados na *cache*. Os outros 750 valores de dados são fornecidos ao processador pela memória principal ou memória de disco. Quais são a *hit rate* e *miss rate* para a *cache*?

Solução: A *miss rate* é $750/2000 = 0.375 = 37.5\%$. A *hit rate* é $1250/2000 = 0.625 = 1 - 0.375 = 62.5\%$.

Princípios Básicos de Cache

- Dois Problemas:
 - Como sabemos se um item de dados está na cache?
 - Se estiver, como encontrá-lo?

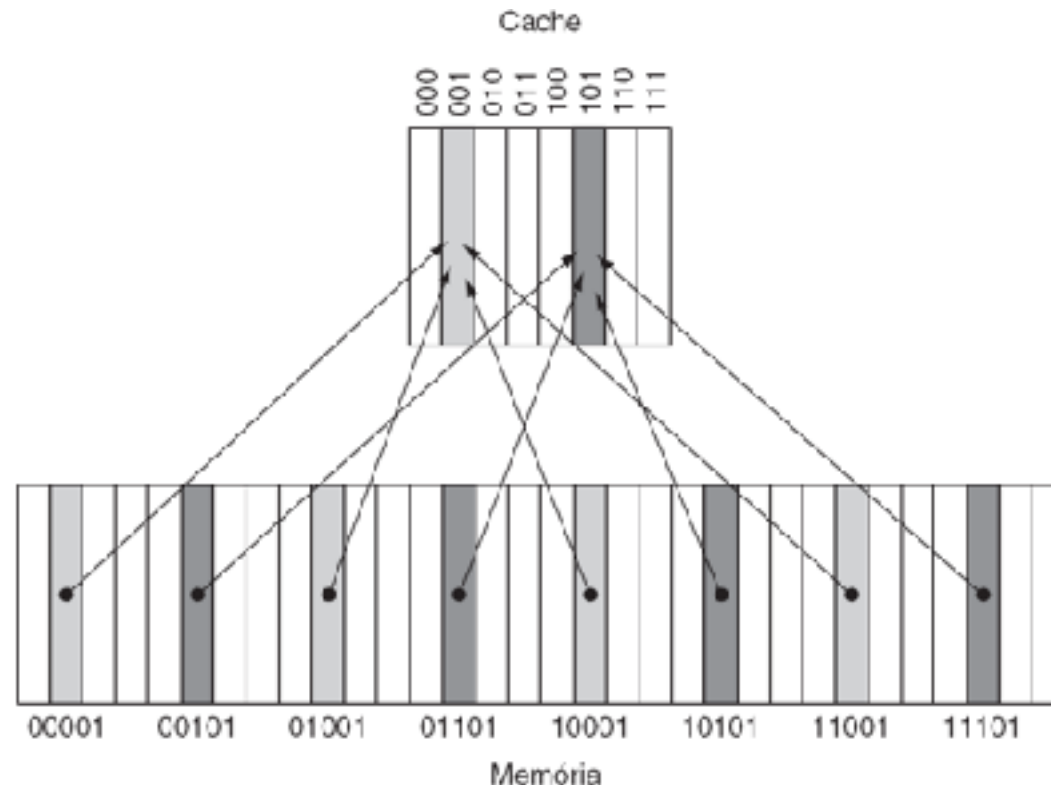
- Primeiro exemplo simples:
 - Processador requisita 1 word : Bloco da cache de 1 word
 - Mapeamento Direto.

Processador requisita X_n

Antes X_n	Depois X_n
X_4	X_4
X_1	X_1
X_{N-1}	X_{N-1}
X_2	X_2
	X_N
X_3	X_3

Exemplo:

- Cache diretamente mapeada com 8 entradas e memória de 32 posições.



- Como saber se uma determinada word está na cache?

TAGs



Informações Auxiliares

- **Tags:** parte do endereço que identifica o bloco que está presente na entrada da *cache*
- **Bit de Validade:** indica se a entrada da *cache* contém um dado válido. As entradas são inválidas se não receberam nenhum dado ainda, por exemplo.

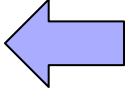
Exemplo de acesso à cache com TAGs - Leitura

Índice	V	Tag	Dados
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		



Exemplo de acesso à cache com TAGs - Leitura

Acessos:

1. 10110 

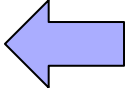
Índice	V	Tag	Dados
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	S	10 _{bin}	Memória (10110 _{bin})
111	N		



Exemplo de acesso à cache com TAGs - Leitura

Acessos:

1. 10110

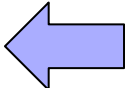
2. 11010 

Índice	V	Tag	Dados
000	N		
001	N		
010	S	11 _{bin}	Memória (11010 _{bin})
011	N		
100	N		
101	N		
110	S	10 _{bin}	Memória (10110 _{bin})
111	N		



Exemplo de acesso à cache com TAGs - Leitura

Acessos:

1. 10110
2. 11010
3. 10110 

Índice	V	Tag	Dados
000	N		
001	N		
010	S	11 _{bin}	Memória (11010 _{bin})
011	N		
100	N		
101	N		
110	S	10 _{bin}	Memória (10110 _{bin})
111	N		



Exemplo de acesso à cache com TAGs - Leitura

Acessos:

1. 10110
2. 11010
3. 10110
4. 11010 ←

Índice	V	Tag	Dados
000	N		
001	N		
010	S	11 _{bin}	Memória (11010 _{bin})
011	N		
100	N		
101	N		
110	S	10 _{bin}	Memória (10110 _{bin})
111	N		



Exemplo de acesso à cache com TAGs - Leitura

Acessos:

1. 10110
2. 11010
3. 10110
4. 11010
5. 10000 ←

Índice	V	Tag	Dados
000	S	10 _{bin}	Memória (10000 _{bin})
001	N		
010	S	11 _{bin}	Memória (11010 _{bin})
011	N		
100	N		
101	N		
110	S	10 _{bin}	Memória (10110 _{bin})
111	N		



Exemplo de acesso à cache com TAGs - Leitura

Acessos:

1. 10110
2. 11010
3. 10110
4. 11010
5. 10000
6. 00011 ←

Índice	V	Tag	Dados
000	S	10 _{bin}	Memória (10000 _{bin})
001	N		
010	S	11 _{bin}	Memória (11010 _{bin})
011	S	00 _{bin}	Memória (00011 _{bin})
100	N		
101	N		
110	S	10 _{bin}	Memória (10110 _{bin})
111	N		



Exemplo de acesso à cache com TAGs - Leitura

Acessos:

1. 10110
2. 11010
3. 10110
4. 11010
5. 10000
6. 00011
7. 10000 ←

Índice	V	Tag	Dados
000	S	10 _{bin}	Memória (10000 _{bin})
001	N		
010	S	11 _{bin}	Memória (11010 _{bin})
011	S	00 _{bin}	Memória (00011 _{bin})
100	N		
101	N		
110	S	10 _{bin}	Memória (10110 _{bin})
111	N		

Exemplo de acesso à cache com TAGs - Leitura

Acessos:

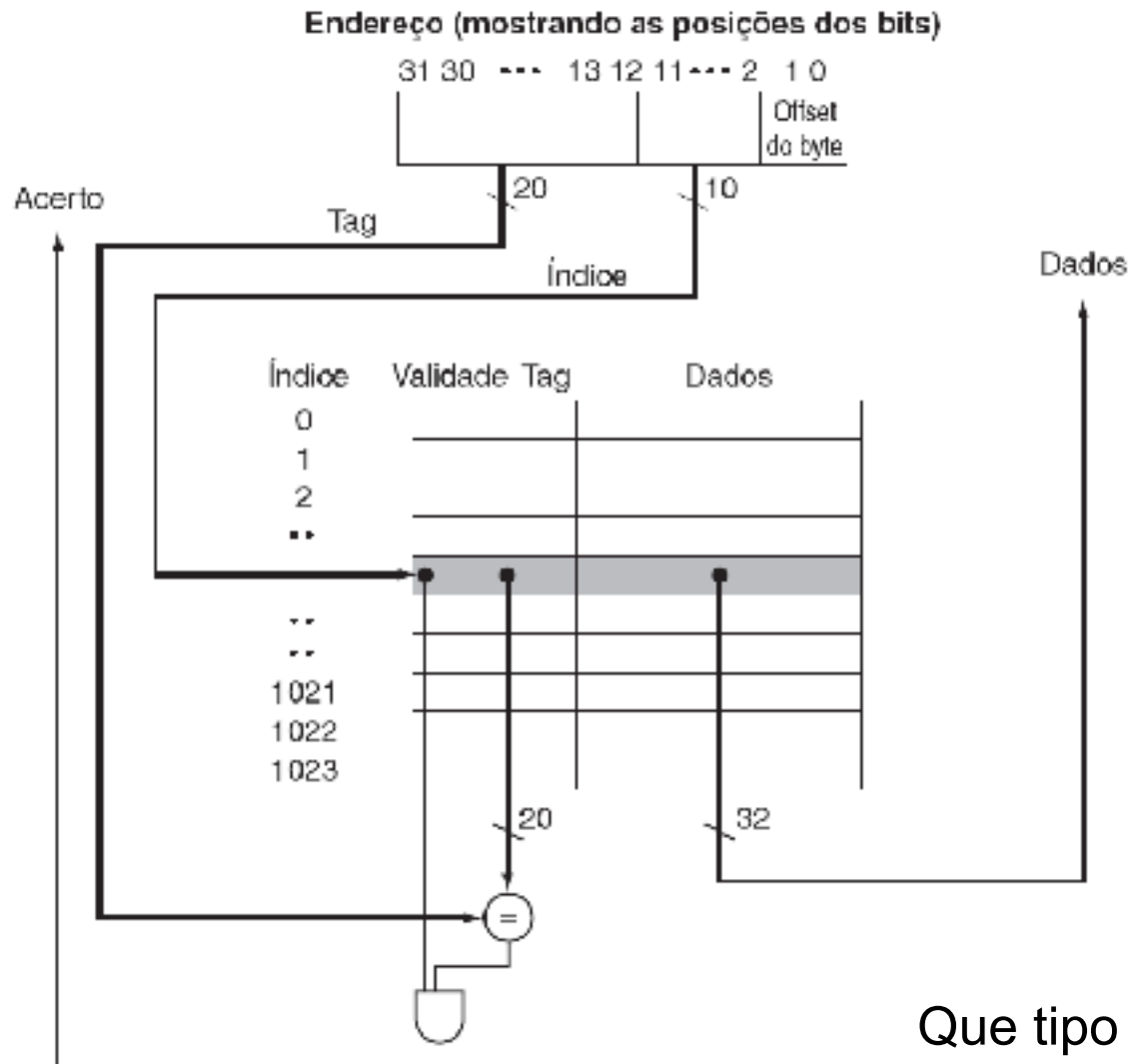
1. 10110
2. 11010
3. 10110
4. **11010**
5. 10000
6. 00011
7. 10000
8. **10010** ←

Índice	V	Tag	Dados
000	S	10 _{bin}	Memória (10000 _{bin})
001	N		
010	S	10 _{bin}	Memória (10010 _{bin})
011	S	00 _{bin}	Memória (00011 _{bin})
100	N		
101	N		
110	S	10 _{bin}	Memória (10110 _{bin})
111	N		

Colisão/Conflito

Ex. real

- No caso RV32I: Endereço 32 bits e cache de 1k



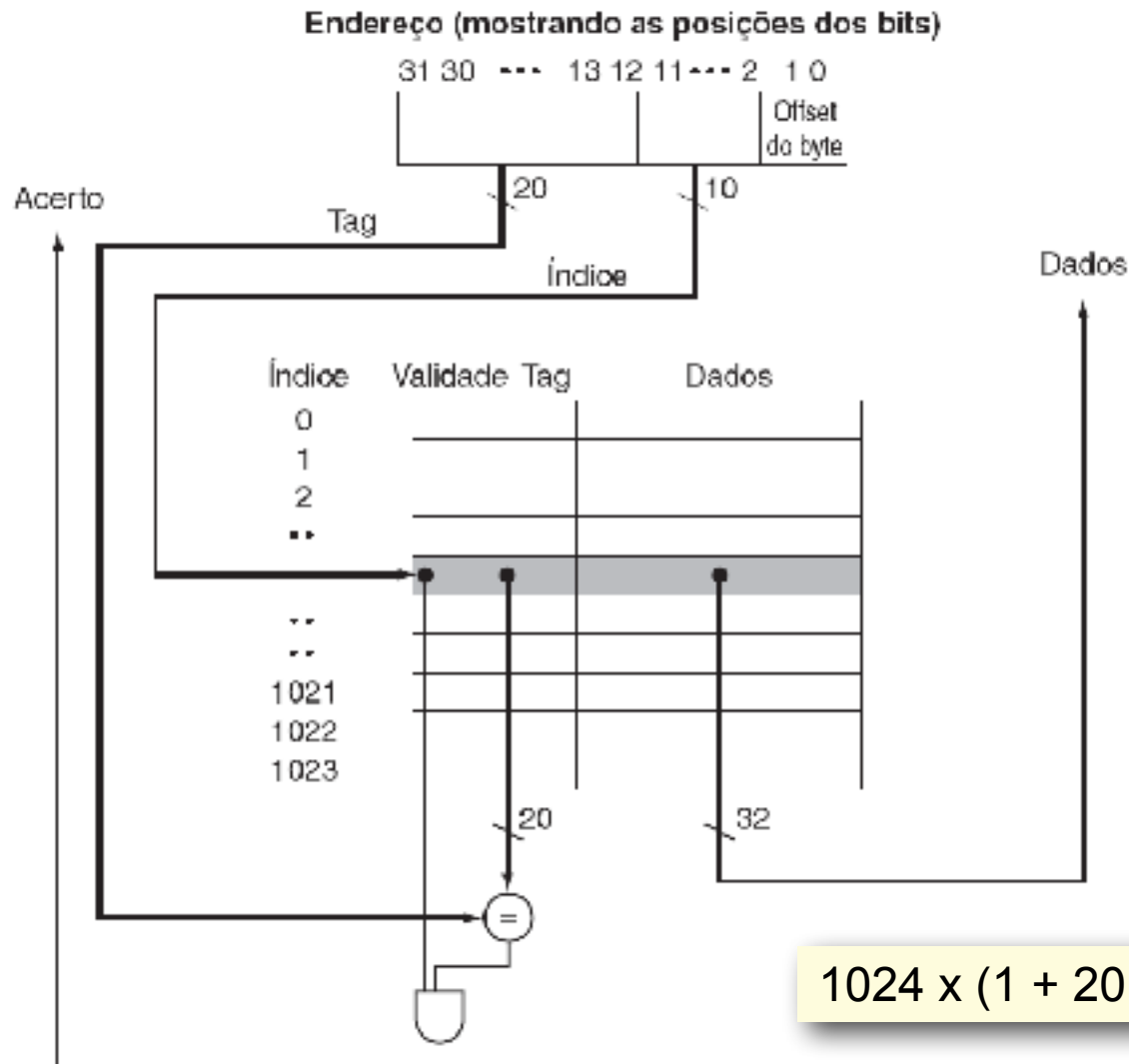
Tamanho real
Tamanho convencional

Que tipo de localidade??



Ex. real

■ Tamanho da Cache ?



$$1024 \times (1 + 20 + 32) = 44.032 \text{ bits}$$



Ex.:

- Considere uma cache com 64 blocos e um tamanho de bloco de 16 bytes. Para qual número de bloco o endereço em bytes 1200 é mapeado?



Ex.:

- Considere uma cache com 64 blocos e um tamanho de bloco de 16 bytes. Para qual número de blocos o endereço em bytes 1200 é mapeado?

Solução: O endereço 1200 corresponde ao bloco $1200/16 = 75$ da memória.

Como a cache possui 64 blocos, $75 \bmod 64 = 11$

Logo os endereços 1200 a 1215 serão mapeados no bloco de endereço 11 da cache.



Ex.:

- Quantos bits no total são necessários para uma cache diretamente mapeada com 16kB de dados e blocos de 4 words, considerando endereçamento de 32 bits?



Ex.:

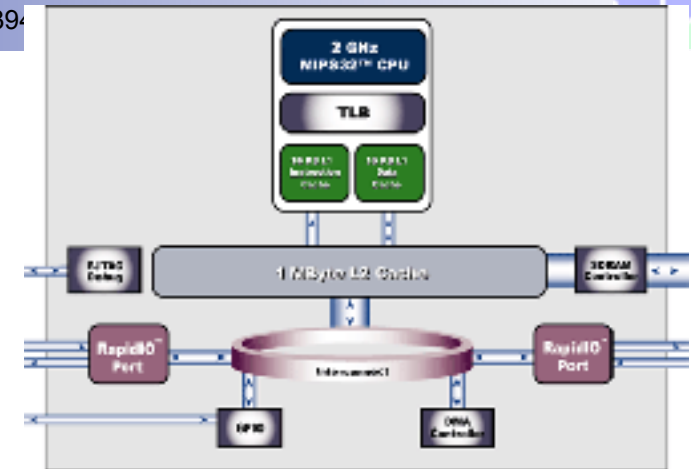
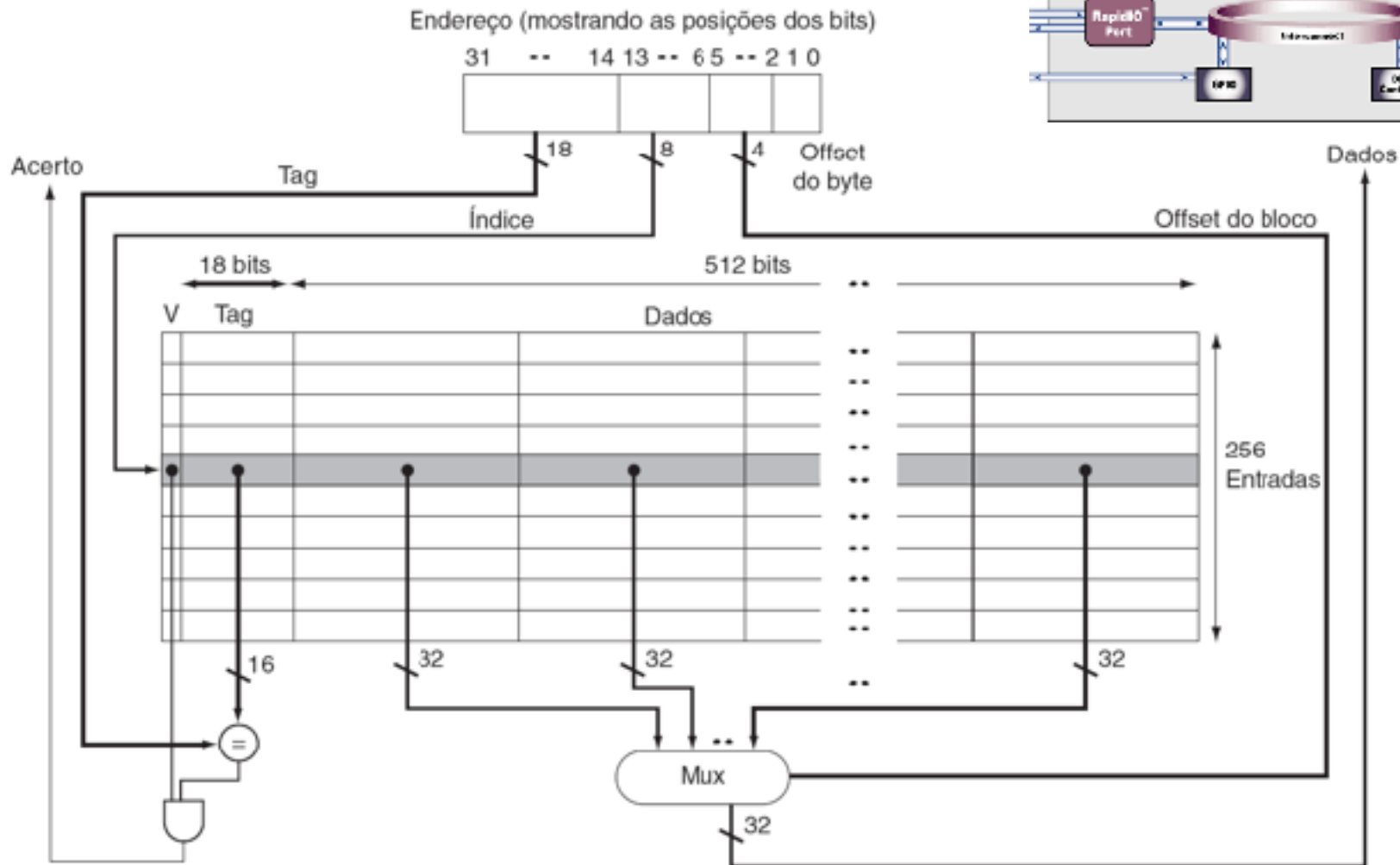
- Quantos bits no total são necessários para uma cache diretamente mapeada com 16kiB de dados e blocos de 4 words, considerando endereçamento de 32 bits?

Solução: $2^{10} \times (4 \times 32 + (32 - 10 - 2 - 2) + 1) = 2^{10} \times 147 = 147 \text{ kibits}$

Ou 1,1484 vezes o necessário que apenas para o armazenamento dos dados.

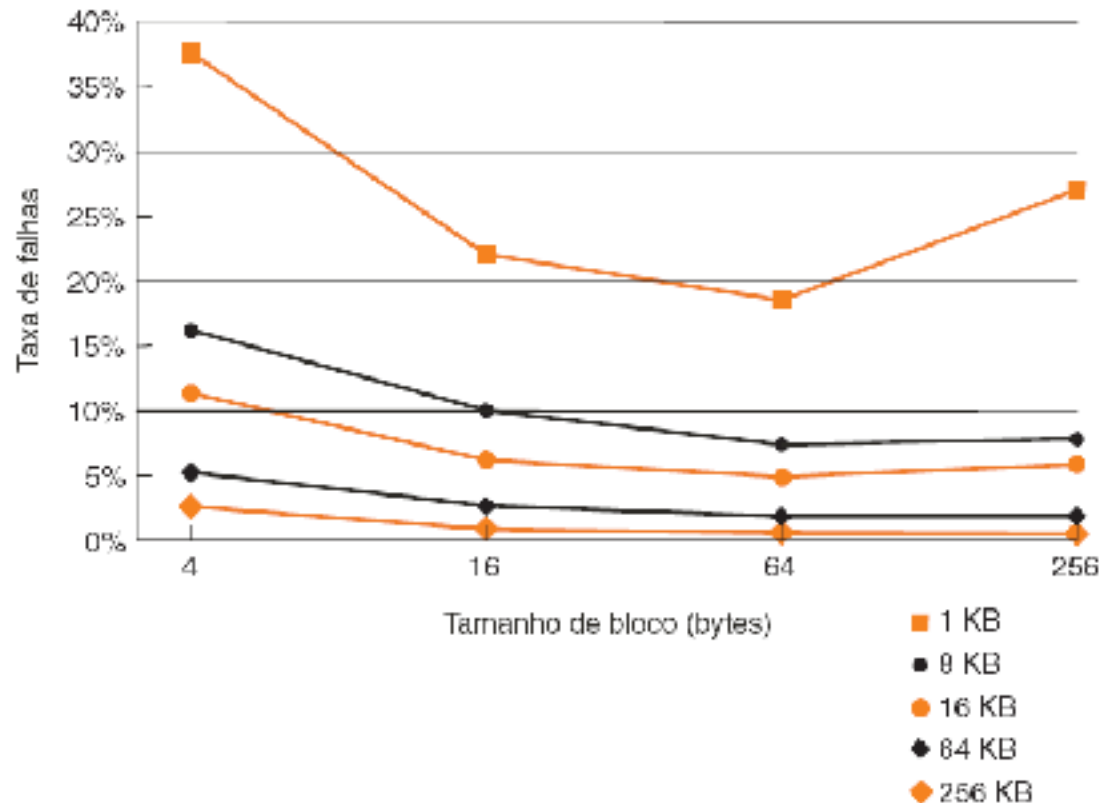
Ex.: MIPS FastMATH

- Cache 16kiB, 256 blocos de 16 words





- Localidade espacial: explorada pelo tamanho do bloco
- Aumentar o tamanho do bloco geralmente diminui a taxa de falhas
- Porém: se tamanho do bloco for significativo perto do tamanho da cache...





Tratando Falhas de Leitura de Cache

- Em caso de acerto:
 - É isso que queremos!!!
- Em caso de falha:
 - Controlador da *cache* deve detectar a falha de leitura e requisitar os dados da memória (ou *cache* de nível inferior).
 - A demanda de tempo para acessar a memória cresce quanto maior for o tamanho do bloco.
 - No pipeline, uma falha causa *stall*, espera até os dados corretos serem enviados pelo controlador à cache e ao processador.
 - Os processos são idênticos para falhas na *cache* de instruções ou dados.



Tratando Escritas na cache

- Apenas na cache de dados.
- Se escrevemos apenas na *cache* > Inconsistência da *cache*. (Cache e memória com dados diferentes)
- Acertos na Escrita:
 - *Write-through* : Sempre escrever na cache E na memória. Simples, mas ineficiente.
 - Outros esquemas mais complexos são adotados. Ex.: Uso de buffers e *Write-Back* (escreve apenas na *cache* e escreve o bloco na memória quando o mesmo for substituído).
- Falhas na Escrita:
 - Lêem o bloco inteiro para a *cache* e depois escrevem a *word*



Desempenho

■ Modelo Simplificado:

Tempo Execução = (ciclos de execução + ciclos stall) x T

Ciclos de Stall = N.Acessos x Taxas de Falhas x
Penalidade da Falha

Pode-se separar em *stall* de leitura e *stall* de escritas.

Melhorar o desempenho:

- Reduzir Taxa de Falhas (como?)
- Reduzir a Penalidade da Falha (como?) (L1, L2, L3..)

Exemplo: Falhas de Instruções e Dados

- Suponha que uma taxa de falhas de cache de instruções para um programa seja de 2% e que uma taxa de falhas de cache de dados seja de 4%. Se um processador possui uma CPI de 2 sem qualquer *stall* de memória e a penalidade de falha é de 100 ciclos para todas as falhas, determine o quanto mais rápido um processador executaria com uma cache perfeita que nunca falhasse.

Considere o programa com 36% acesso à memória de dados.



Exemplo: Falhas de Instruções e Dados

- Suponha que uma taxa de falhas de cache de instruções para um programa seja de 2% e que uma taxa de falhas de cache de dados seja de 4%. Se um processador possui uma CPI de 2 sem qualquer *stall* de memória e a penalidade de falha é de 100 ciclos para todas as falhas, determine o quanto mais rápido um processador executaria com uma cache perfeita que nunca falhasse.

Considere o programa com 36% de instruções de acesso à memória de dados.

Solução:

Ciclos de falha de instrução = $I \times 0,02 \times 100 = 2xI$

Ciclos de falha de dados = $I \times 0,36 \times 0,04 \times 100 = 1,44xI$

Logo: ciclos de stall = $2xI + 1,44xI = 3,44xI$

$$\frac{\text{Tempo com Stall}}{\text{Tempo perfeito}} = \frac{I(3,44 + 2)T}{I2T} = 2,72$$

Ex.2: Alterando a frequência de clock

- Suponha que aumentemos o desempenho do processador do exemplo anterior dobrando a frequência do clock. Como a frequência da memória principal é improvável de ser alterada, considere que o tempo absoluto para manipular uma falha de cache não mude. O quanto mais rápido será o computador com maior frequência de clock, considerando a mesma taxa de falhas do exemplo anterior?

Ex.2: Alterando a frequência de clock

- Suponha que aumentemos o desempenho do processador do exemplo anterior dobrando a velocidade do clock. Como a velocidade da memória principal é improvável de ser alterada, considere que o tempo absoluto para manipular uma falha de cache não mude. O quanto mais rápido será o computador com maior frequência de clock, considerando a mesma taxa de falhas do exemplo anterior?

Solução: Com os ciclos mais rápidos a nova penalidade de falha será o dobro de ciclos de clock, 200 ciclos por falha.

Tempo de ciclos de falha por instrução

$$= (0,02 \times 200) + 0,36 \times (0,04 \times 200) = 6,88$$

Logo o computador mais rápido com falhas de cache terá um

CPI de $2 + 6,88 = 8,88$ (anterior era 5,44), logo

$$\frac{\text{Tempo}_{\text{clock lento}}}{\text{Tempo}_{\text{clock rápido}}} = \frac{I \times 5,44 \times T}{I \times 8,88 \times \frac{T}{2}} = 1,23 \quad \text{E não 2!!!!}$$

Ex.3: Caches multinível

- Suponha que tenhamos um processador com CPI básico de 1,0, considerando que todas as referências acertem na cache primária e uma frequência de clock de 5GHz. Considere um tempo de acesso à memória principal de 100 ns, incluindo todo tratamento de falhas. Suponha que a taxa de falhas por instrução da cache primária seja de 2%. O quanto mais rápido será o processador se acrescentarmos uma cache secundária que tenha um tempo de acesso de 5 ns para um acerto ou uma falha e que seja grande o suficiente para reduzir a taxa de falhas para a memória principal para 0,5%?



Ex.3: Caches multinível

- Suponha que tenhamos um processador com CPI básico de 1,0, considerando que todas as referências acertem na cache primária e uma frequência de clock de 5GHz. Considere um tempo de acesso à memória principal de 100ns, incluindo todo tratamento de falhas. Suponha que a taxa de falhas por instrução da cache primária seja de 2%. O quanto mais rápido será o processador se acrescentarmos uma cache secundária que tenha um tempo de acesso de 5ns para um acerto ou uma falha e que seja grande o suficiente para reduzir a taxa de falhas para a memória principal para 0,5%?

Solução: Penalidade de falhas da memória principal: $100\text{ns}/0,2\text{ns} = 500$ ciclos

CPI efetivo com um nível de cache: $\text{CPI total} = \text{CPI básico} + \text{Ciclos Stall por instr}$

$$\text{CPI total} = 1,0 + 0,02 \times 500 = 11,00$$

Penalidade de falhas no 2º nível: $5\text{ns}/0,2\text{ns} = 25$ ciclos

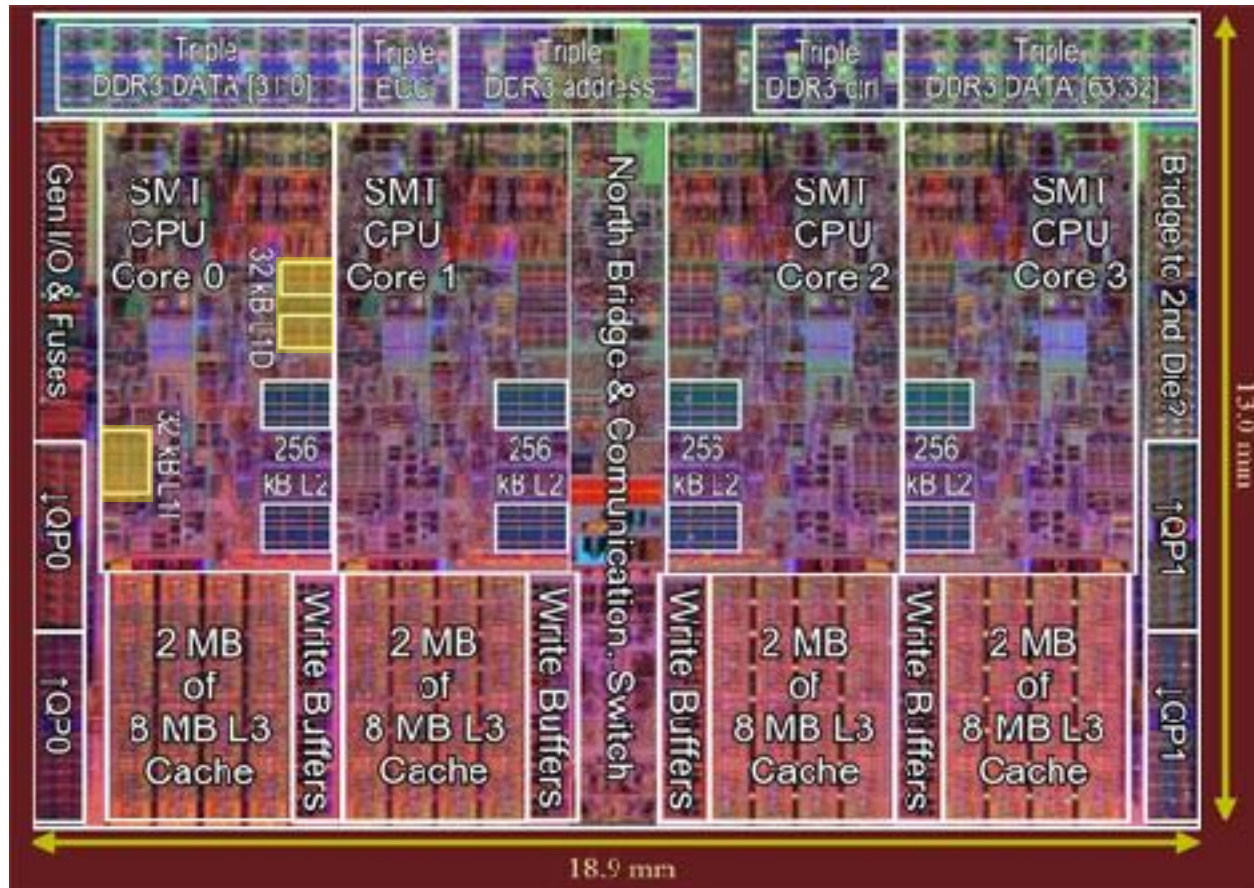
CPI efetivo com 2 níveis $\text{CPI total} = \text{CPI básico} + \text{Stall primário} + \text{Stall secundário}$

$$\text{CPI total} = 1,0 + 0,02 \times 25 + 0,005 \times 500 = 1 + 0,5 + 2,5 = 4,0$$

Logo: $11,0/4,0 = 2,8$ vezes mais rápida!

Caches multinível

Ex.: Intel i7 primeira geração

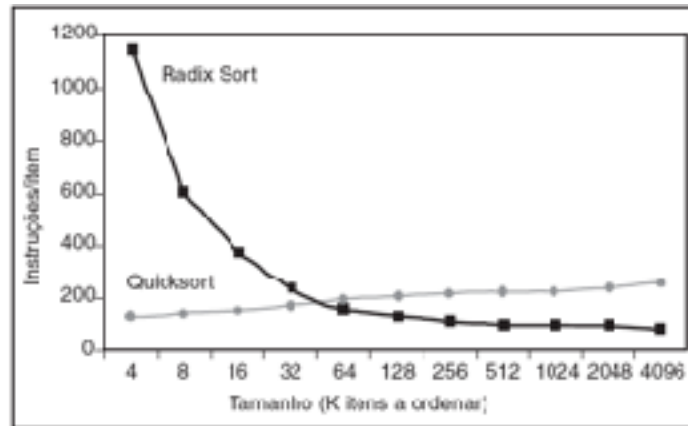


Cache L1 dividida: 32KiB I-cache, 32KiB D-cache,
 Cache L2 unificada: 512KiB
 Cache L3 compartilhada: 8MiB

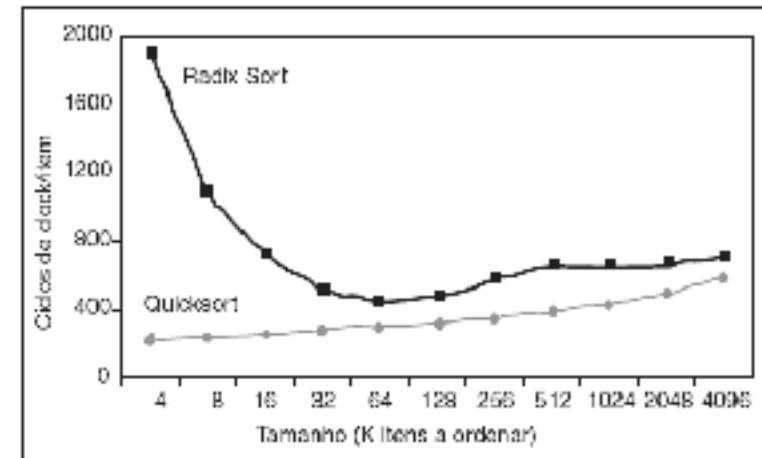


Obs.: Quicksort x Radix Sort

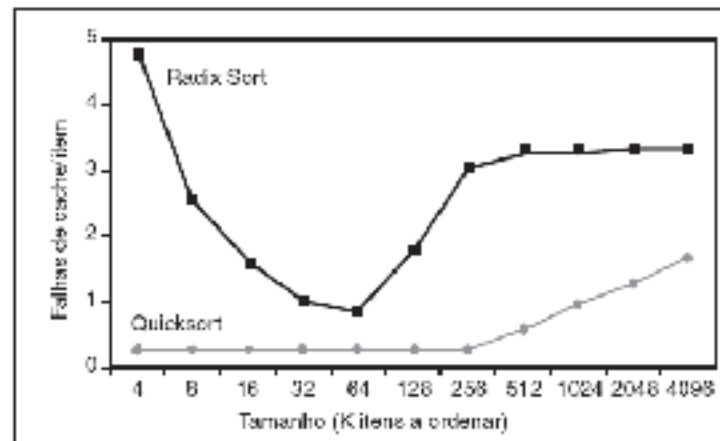
■ Teórico



■ Observado: Tempo por item

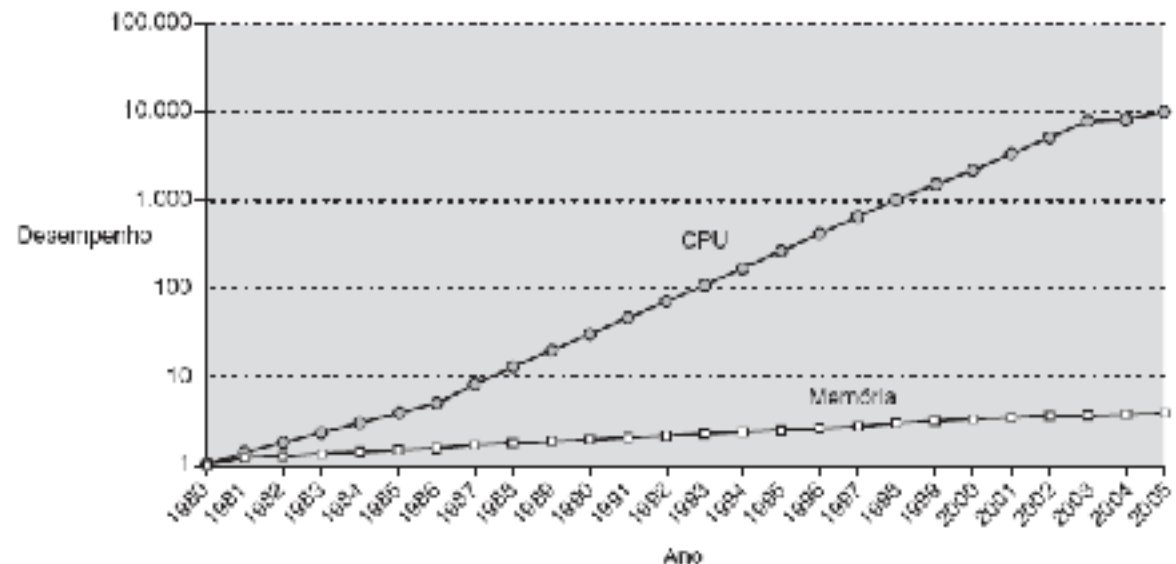


■ Muito mais falhas por item



Conclusões

- A análise algorítmica padrão ignora o impacto da hierarquia da memória (Falhas) e do Pipeline (Hazards).
- Entender o comportamento da hierarquia da memória, ILP (Pipeline) e multiprocessamento é fundamental para compreender o desempenho dos programas nos computadores atuais.
 - **As velocidades do processador continuam a aumentar muito rápido**
 - **muito mais rápido do que os tempos de acesso à DRAM ou ao disco**



- **Desafio de projeto: lidar com essa crescente disparidade**